Bharati Vidyapeeth

(Deemed to be University)

Department of Engineering and Technology
Navi Mumbai

PROJECT BASED LEARNING

STUDENT ATTENDANCE MANAGEMENT SYSTEM

is submitted to

Department of Computer Science Engineering,

In partial fulfilment of

SOFTWARE TESTING AND QUALITY ASSURANCE

Submitted To:

Prof. Dhanashree Waikar

Submitted By:

Roll no.	Name of Student	PRN No.
04	Aditya Bafna	2443110010
07	Adwai Singh	2443110006
16	Aryan Singh	2443110063

BHARATI VIDYAPEETH (DEEMED TO BE UNIVERSITY)
DEPARTMENT OF ENGINEERING & TECHNOLOGY
Sector 3-A, Plot No. KC-1, Kharghar, Navi Mumbai - 410210
REACCREDITED WITH 'A++' BY NAAC

VISION OF THE INSTITUTE:

To be World Class Institute for Social Transformation Through Dynamic Education.

MISSION OF THE INSTITUTE:

- To provide quality technical education with advanced equipment, qualified faculty members, infrastructure to meet needs of profession and society.
- To provide an environment conducive to innovation, creativity, research, and entrepreneurial leadership.
- To practice and promote professional ethics, transparency and accountability for social community, economic and environmental conditions.

VISION OF THE DEPARTMENT

To be focused on innovative and quality education in computer science and engineering that prepares professionals for development of society.

MISSION OF THE DEPARTMENT

- To provide academic environment for the development of skilled professionals
- To cultivate research culture that contributes to the sustainable development of the society.
- To enhance academic and industry collaborations for global exposure.

PROGRAM OUTCOMES

1. **PO1 :** Engineering Knowledge
   Apply the knowledge of mathematics, natural science, computing, and engineering fundamentals and an engineering specialisation to the solution of complex engineering problems.
2. **PO2:** Problem Analysis
   Identify, formulate, and analyse complex engineering problems, reaching substantiated conclusions with consideration for the holistic nature of the problem.
3. **PO3:** Design/Development of Solutions

Design creative solutions for complex engineering problems and design systems, components, or processes to meet identified needs — with consideration for public health and safety, and cultural, societal, and environmental factors. Sustainability is now intrinsic to design.

4. **PO4:** Conduct Investigations of Complex Problems

   Conduct investigations of complex engineering problems using research-based knowledge and research methods, including design of experiments, analysis and interpretation of data, and synthesis of information to provide valid conclusions.

5. **PO5:** Engineering Tool Usage

   Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools, including prediction and modelling — recognising their limitations — to solve complex engineering problems. (Previously titled "Modern Tool Usage.")

6. **PO6:** The Engineer and the World

   Analyse and evaluate societal and environmental aspects while solving complex engineering problems, considering their impact on sustainability with reference to economy, health, safety, legal frameworks, culture, and the environment.

7. **PO7:** Ethics

   Apply ethical principles and commit to professional ethics, human values, diversity, and inclusion; and adhere to relevant national and international laws. (Diversity and inclusion are now explicit additions.)

8. **PO8:** Individual and Collaborative Team Work

   Function effectively as an individual, and as a member or leader in diverse and multi-disciplinary teams.

9. **PO9:** Communication

   Communicate effectively and inclusively within the engineering community and with society at large — comprehending and writing effective reports and design documentation, and making effective presentations — considering cultural, language, and learning differences.

10. **PO10:** Project Management and Finance

    Apply knowledge and understanding of engineering management principles and economic decision-making, and apply these to one's own work, as a member and leader in a team, to manage projects in multi-disciplinary environments.

11. **PO11:** Life-Long Learning

    Recognise the need for, and have the preparation and ability for, independent and life-long learning, adaptability to new and emerging technologies, and critical thinking in the broadest context of technological change.

PROGRAM EDUCATIONAL OBJECTIVES

The students of B.TECH. (Computer Science and Engineering), after graduating will able to:

1. Demonstrate technical and professional competencies by applying Engineering Fundamentals, knowledge of computing and technologies.
2. Exhibit effective personality, good communication and team building skills
3. Adopt to the latest trends in the field of computer science and engineering.

PROGRAM SPECIFIC OUTCOMES

1. To design, develop and implement computer programs on hardware towards solving problems.
2. To employ expertise and ethical practice through continuing intellectual growth and adapting to the working environment.

SOFTWARE TESTING & QUALITY ASSURANCE COURSE OUTCOMES:

CO#	Course Outcome
CO1	Understand and apply software testing fundamentals, life-cycle models, and quality assurance principles.
CO2	Design effective test cases using black-box techniques (Equivalence Class Partitioning and Boundary Value Analysis). constraints
CO3	Implement white-box and code-level verification techniques including path testing and assertion coverage.
CO4	Formulate comprehensive IEEE 829 Software Test Plans and detailed Test Procedure documents for real systems.
CO5	Perform multi-level testing execution spanning Unit, Integration, Regression, and System verification.
CO6	Analyze test execution results, evaluate software reliability, calculate metrics, and manage quality assurance. emerging trends shaping the future of AI

SDG mapped to Project topic: SDG 4 — Quality Education (Target 4.1 & 4.a: Digital tools ensuring academic continuity, fair evaluation, and auditable governance)

BHARATI VIDYAPEETH (DEEMED TO BE UNIVERSITY)
DEPARTMENT OF ENGINEERING & TECHNOLOGY
Sector 3-A, Plot No. KC-1, Kharghar, Navi Mumbai - 410210
REACCREDITED WITH 'A++' BY NAAC

Certificate

Certified that this project report "STUDENT ATTENDANCE MANAGEMENT SYSTEM" is the bonafide work of "Aditya Bafna, Adwai Singh, Aryan Singh" who carried out the project work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Prof. Dhanashree Waikar
SUPERVISOR

Dr. Nidhi Sharma
HEAD OF THE DEPARTMENT

INDEX

Sr. No.	Table Of Content	Page No
1. 1	Abstract	6
2. 2	Introduction (Topic, Methodology, Motivation & STQA Lifecycle)	7
3. 3	Literature Review (Comparative Analysis & Quality Standards)	7
4. 4	Theory & Test Planning (MVC Design, IEEE 829 Plan & Procedures)	8
5. 5	Implementation Details (Modular Client Architecture & Test Engine)	9
6. 6	Results with Screenshots (Unit, Integration, Regression, System Verification)	10
7. 7	Conclusion & Future Scope	14
8. 8	References & Bibliography (IEEE Format)	14

TABLE OF FIGURES

Figure 1 Teacher Authentication and Login Portal..... Page 11
Figure 2 Attendance Dashboard and Classroom Metrics..... Page 11
Figure 3 Student Management and Roster Registry ..... Page 11
Figure 4 Daily Attendance Marking Interface ..... Page 11
Figure 5 Attendance Reports and Eligibility View ..... Page 12
Figure 6 STQA Automated Unit Testing Suite Output..... Page 12
Figure 7 STQA Integration Testing Suite Output..... Page 12
Figure 8 STQA Regression Testing Suite Output..... Page 13
Figure 9 STQA System End-to-End Testing Output ..... Page 13

Abstract

The Student Attendance Management System (SAMS) is an academic web application engineered from scratch using pure HTML5, CSS3, Vanilla JavaScript (ES6+), and browser

localStorage for client-side persistence. Designed to serve classroom educators, the system automates student enrollment, daily date-specific attendance marking, retroactive session editing, cumulative percentage computation, and academic eligibility auditing against the university's mandatory 75.00% threshold. As an academic Software Testing and Quality Assurance (STQA) Project-Based Learning initiative, a multi-tiered testing strategy was implemented. A test suite of 31 automated test cases was executed across four testing levels: Unit Testing (15 cases including Boundary Value Analysis at 0%, 74.99%, 75.00%, 75.01%, and 100%), Integration Testing (6 cases), Regression Testing (5 cases), and System Testing (5 complete end-to-end workflows). The application achieved a 100% test pass rate with zero defect regressions, proving deterministic mathematical precision and data integrity.

Introduction

Description of Topic

Classroom attendance monitoring is an essential academic requirement across higher education institutions. Bharati Vidyapeeth mandates a minimum 75.00% attendance threshold for examination eligibility. Traditional physical attendance registers are prone to manual miscalculation, difficult to audit, and lack real-time eligibility warnings. SAMS provides a zero-dependency, reliable local web application that automates roster management, date-indexed attendance marking, and real-time eligibility analytics.

Methodology & STQA Quality Engineering Approach

Quality was engineered into SAMS through a structured testing lifecycle: (1) Requirements Specification with deterministic mathematical rules; (2) IEEE 829 Test Plan and Test Procedure authoring; (3) In-browser automated STQA Test Runner executing assertions with millisecond precision; and (4) Four-level verification covering Unit, Integration, Regression, and System tests.

Motivation to Choose the Topic

The motivation behind this project is twofold: First, delivering a clean, lightweight web utility that operates locally in any browser without requiring complex server stacks or database installations. Second, designing an application whose core 75% eligibility rule serves as an exemplary testbed for Boundary Value Analysis (BVA), Equivalence Class Partitioning (ECP), and regression verification.

Literature Review

Contemporary institutional attendance solutions generally fall into three architectural categories: heavy server-side ERPs, biometric/RFID hardware appliances, and client-side web utilities. Myers et al. [1] emphasize that software quality assurance must begin at requirement formulation and proceed through rigorous specification-based black-box and white-box test design. Jorgensen [4] further demonstrates that decision-critical thresholds, such as academic eligibility cuts, inherently concentrate defects at boundary conditions (e.g., 74.99% vs 75.00%). Existing heavyweight ERPs introduce significant latency, complex database dependencies, and lack transparent in-browser regression testing harnesses. In contrast, SAMS adopts the IEEE 829 [3] test documentation framework alongside lightweight browser-native persistence, ensuring deterministic calculation, sub-millisecond assertion execution, and auditable academic integrity without external server vulnerabilities.

Theory

Software Architectural Design

SAMS follows a client-side Model-View-Controller (MVC) pattern. The Model layer manages localStorage schemas, the View layer renders semantic HTML5/CSS3 templates, and the Controller layer (auth.js, students.js, attendance.js, reports.js) encapsulates business logic and assertion testing.

Mathematical Model & 75% Eligibility Policy

For total conducted classes T, attended classes P, and absent classes A (T = P + A):

Formula: Percentage (Pct) = (P / T) * 100 (when T > 0; Pct = 0.00% when T = 0 to prevent division-by-zero).

Decision Rule: If Pct >= 75.00% -> 'Eligible'; If Pct < 75.00% -> 'Short Attendance'. Boundary conditions evaluated: 0%, 74.99%, 75.00%, 75.01%, 100%.

Software Test Plan Document (IEEE 829 Standard)

The formal IEEE 829 Test Plan defines scope, strategy, and criteria for system verification:

No.	Test Plan Attribute	Specification Details
1	Introduction & Objective	Formal quality verification of SAMS across all modules, math formulas, and persistence layers.
2	Scope of Testing	Covers authentication, student CRUD, daily attendance marking, reports, and boundary math.
3	Features Not Tested	Cloud databases, third-party payment gateways, multi-tenant auth (outside architecture).
4	Testing Strategy & Levels	Hybrid Black-Box & White-Box testing across Unit, Integration, Regression, and System tiers.
5	Test Environment	Microsoft Edge / Google Chrome, HTML5 LocalStorage API, Windows 10/11 OS, 4GB RAM.
6	Test Data Specification	Admin credentials, Student Roster (Rolls 101-106, Div A/B), 4 historical attendance dates.
7	Entry & Exit Criteria	Entry: Functional modules complete. Exit: 100% test execution, 100% pass rate, 0 defects.
8	Roles & Responsibilities	Project Team (Aditya, Adwai, Aryan): Design, implementation, test scripting, and report authoring.
9	Risks & Mitigations	Risk: LocalStorage wipe. Mitigation: Built-in automated seed fallback re-initialization.
10	Deliverables & Criteria	Test Plan, Test Procedures, Automated Test Suite (tests.js), Live Screenshots, Final Report.

Software Test Procedure Document

Step-by-step test procedures for primary system workflows:

ID	Procedure Title	Step-by-Step Procedure	Expected Result	Actual Result	Status
TP-01	Teacher Login	1. Open index.html 2. Enter admin / admin123 3. Click Sign In	Redirect to dashboard.html with active session	Session stored; redirected to dashboard	PASS
TP-02	Empty Input Check	1. Leave fields blank 2. Click Sign In	Validation message: 'Please enter both username and password.'	Validation alert displayed; navigation blocked	PASS
TP-03	Student Enrollment	1. Open students.html 2. Enter 107, Aarav, Div A 3. Click Add Student	Student 107 added to roster table and localStorage	Success alert displayed; table updated	PASS
TP-04	Duplicate Roll No	1. Enter existing roll 101 2. Submit enrollment	Error alert: 'Roll number 101 already exists.'	Duplicate prevented; record rejected	PASS
TP-05	Student Deletion	1. Locate student 198 2. Click Delete & confirm	Student removed from roster and attendance records cascaded	Record deleted; attendance history purged	PASS
TP-06	Mark Attendance	1. Select 2026-10-05 2. Mark 101 Present, 102 Absent 3. Click Save	Attendance serialized to localStorage under date key	Success alert; date added to history chips	PASS
TP-07	Update Attendance	1. Load existing date 2026-10-01 2. Change status & save	Existing date record updated in-place without duplicate keys	Record updated; reports recalculated	PASS
TP-08	75% Boundary Test	1. Test 3 present out of 4 classes 2. Check eligibility	Percentage: 75.00%, Status: 'Eligible'	Evaluated to 75.00% Eligible	PASS
TP-09	Filter Reports	1. Open reports.html 2. Filter by 'Eligible' and 'Div A'	Table filters to show only matching students	Matching records displayed with badges	PASS
TP-10	End-to-End Flow	1. Login -> Add student 120 -> Mark 2/2 -> Verify report	Student 120 displays 100.00% and 'Eligible' status	Complete workflow succeeded seamlessly	PASS

Implementation details

Technology Stack & Directory Structure

Built entirely from scratch using HTML5, CSS3, Vanilla JavaScript (ES6+), and browser localStorage without external frameworks or CDNs.

```
Student-Attendance-System/
├── index.html / auth.js        <- Login Portal & Session Authentication
├── dashboard.html             <- Executive Metrics & Quick Classroom Actions
├── students.html / students.js  <- Student Enrollment (Regex & Duplicate Prevention)
├── attendance.html / attendance.js <- Daily Date-Indexed Marking Sheet
├── reports.html / reports.js    <- Cumulative Statements & Eligibility Filters
├── tests.html / tests.js      <- Interactive STQA Automated Test Harness
└── css/style.css              <- Responsive Academic Theme & Status Badges
```

Core Functional Modules

- **auth.js:** Validates credentials ('admin' / 'admin123'), trims inputs, and guards routes.
- **students.js:** Validates roll numbers (alphanumeric), names (letters/spaces only), enforces uniqueness, and cascades deletions.
- **attendance.js:** Handles YYYY-MM-DD date selection, status toggles (Present/Absent), and retroactive editing.
- **reports.js:** Computes exact percentage math rounded to 2 decimals, evaluates 75% threshold, and provides filters.
- **tests.js:** Executes all 31 automated tests with microsecond execution latency and assertion tracking.

Results (with screenshots)

Empirical verification results from live test runner execution and application operation (All 31 test cases passed with 100% success rate):

Unit Testing Output (UT01 - UT15)

ID	Module	Description	Test Input	Actual Outcome	Status
UT01	Authentication	Verify login succeeds with valid username and password	{"username":"admin","password":"admin123"}	success: true, message: "Login successful. Redirecting..."	PASS
UT02	Authentication	Verify login fails when fields are blank or whitespace	{"username":" ","password":""}	success: false, message: "Please enter both username and password."	PASS
UT03	Student Management	Verify rejection of student name containing invalid symbols/numbers	{"rollNo":"109","name":"John@123","division":"A"}	isValid: false, error: "Name must contain 2-50 characters with letters and spaces only."	PASS
UT04	Student Management	Verify duplicate roll numbers are prevented	{"rollNo":"101","name":"Another Student","division":"A"}	isValid: false, error: "Roll number '101' already exists. Duplicates are not allowed."	PASS
UT05	Student Management	Verify student deletion removes target record and leaves remainder intact	Roll No: 101	List length: 1, Remaining: [102]	PASS
UT06	Reports & Calculation	Verify formula: (Present / Total) * 100 for 15 present out of 20 classes	{"present":15,"total":20}	percentage: 75.00%, raw: 75	PASS
UT07	Reports & Calculation	Verify eligibility status: 16/20 (80%) -> Eligible; 14/20 (70%) -> Short Attendance	Pairs: (16, 20) and (14, 20)	80%: Eligible, 70%: Short Attendance	PASS
UT08	Attendance	Verify marking Present accepts and stores status successfully	{"date":"2026-10-05","statuses":{"101":"Present"}}	success: true, message: "Attendance recorded successfully for 2026-10-05 (1 students)."	PASS
UT09	Attendance	Verify marking Absent accepts and stores status successfully	{"date":"2026-10-05","statuses":{"102":"Absent"}}	success: true, message: "Attendance updated successfully for 2026-10-05 (1 students)."	PASS
UT10	Attendance	Verify system rejects invalid status values (e.g., "Late", "HalfDay")	{"date":"2026-10-05","statuses":{"101":"Late"}}	success: false, message: "Invalid status 'Late' for roll number 101. Must be 'Present' or 'Absent'."	PASS
UT11	Boundary Testing	Verify boundary condition: 0 present out of 10 classes (0.00%)	{"present":0,"total":10}	percentage: 0.00%, status: Short Attendance	PASS
UT12	Boundary Testing	Verify boundary condition: 7499 present out of 10000 classes (74.99%)	{"present":7499,"total":10000}	percentage: 74.99%, status: Short Attendance	PASS
UT13	Boundary Testing	Verify threshold boundary: 3 present out of 4 classes (Exactly 75.00%)	{"present":3,"total":4}	percentage: 75.00%, status: Eligible	PASS

ID	Module	Description	Test Input	Actual Outcome	Status
UT14	Boundary Testing	Verify boundary condition: 7501 present out of 10000 classes (75.01%)	{"present":7501,"total":10000}	percentage: 75.01%, status: Eligible	PASS
UT15	Boundary Testing	Verify upper boundary condition: 10 present out of 10 classes (100.00%)	{"present":10,"total":10}	percentage: 100.00%, status: Eligible	PASS

Integration Testing Output (IT01 - IT06)

ID	Interface	Integration Scenario	Test Input	Actual Outcome	Status
IT01	Auth to Dashboard Integration	Verify login authentication properly initiates and authorizes dashboard session metrics	Valid login -> session verified -> calculateDashboardSummary()	Login: true, Dashboard Total Students: 6, Avg Pct: 59.17%	PASS
IT02	Student to Attendance Integration	Verify newly added student immediately surfaces in attendance registry for marking	Add student 199 ("Test Student IT", Div A)	addSuccess: true, Found in Student Registry: true	PASS
IT03	Attendance Marking & Storage Integration	Verify attendance UI submission writes validated map to persistent storage	Mark 101 Present, 102 Absent for date 2026-10-10	saveSuccess: true, 101: Present, 102: Absent	PASS
IT04	Attendance to Report Integration	Verify updated attendance immediately recalculates student attendance report totals	generateAllReports() with updated records	Report Total: 1, Present: 1, Pct: 100.00%	PASS
IT05	Calculation to Eligibility Integration	Verify attendance records flow through calculation formula directly to correct eligibility classification	Student 103 with 2 classes: 1 Present, 1 Absent	Pct: 50.00%, Status: Short Attendance, isEligible: false	PASS
IT06	Deletion Cascading Integration	Verify deleting a student purges them from the student database and cascaded attendance reports	Add 198 -> Mark attendance -> Delete 198	Delete Success: true, Still in Registry: false	PASS

Regression Testing Output (RT01 - RT05)

ID	Target Module	Refactored Feature	Input / Scope	Post-Refactor Outcome	Status
RT01	Regression: Authentication	Sanitization updates applied to string trimming	Credentials: admin / admin123	success: true, message: "Login successful. Redirecting..."	PASS
RT02	Regression: Student Management	Regex enhancement to support hyphenated names	Name "Mary-Jane Watson", Roll "110", Div "A"	isValid: true, Sanitized Name: "Mary-Jane Watson"	PASS
RT03	Regression: Attendance Engine	Core attendance storage active	Date 2026-10-12, Status: 101 Present, 102 Absent	saveSuccess: true	PASS
RT04	Regression: Calculation Logic	Mathematical logic refactored for edge cases	0 present, 0 total classes	Pct: 0.00%, Status: No Classes, isNaN: false	PASS
RT05	Regression: Eligibility Boundary	Eligibility evaluation function refactored	15 present out of 20 classes (75.00%)	Pct: 75.00%, Status: Eligible	PASS

System Testing Output (ST01 - ST05)

ID	Workflow	End-to-End Scenario	User Input / Journey	System Verification Outcome	Status
ST01	System End-to-End	Execute complete workflow: Login -> Add Student -> Mark Attendance -> Generate Report -> Verify Percentage	User admin logs in, adds student 120, marks 2 present out of 2 classes, checks report	E2E Success: Found 120 with Pct: 100.00%, Status: Eligible	PASS
ST02	System End-to-End	Add multiple students with varying attendance records, verify distinct eligibility categories in report	3 students: Student A (100%), Student B (50%), Student C (75%)	S1: Eligible (100.00%), S2: Short Attendance (25.00%), S3: Eligible (75.00%)	PASS
ST03	System End-to-End	Attempt invalid login, verify error display, then authenticate with correct credentials and reach dashboard	Step 1: wrong_user / wrong_pass; Step 2: admin / admin123	Attempt 1 Success: false (Error: "Invalid username or password. Please try again."), Attempt 2 Success: true	PASS
ST04	System End-to-End	Delete student and verify immediate removal from student registry, attendance views, and reports	Add student 125 -> Save attendance -> Delete student 125 -> Generate report	In Registry: false, In Report: false	PASS
ST05	System End-to-End	Test critical boundary case: Exactly 75.00% attendance in end-to-end report workflow	Student 101 with 3 Present and 1 Absent across 4 recorded dates	Total: 4, Present: 3, Percentage: 75.00%, Status: Eligible	PASS

Comprehensive Test Execution Summary

Testing Level	Suite Range	Total Cases	Passed	Failed	Rate
Unit Testing	UT01 - UT15	15	15	0	100%
Integration Testing	IT01 - IT06	6	6	0	100%
Regression Testing	RT01 - RT05	5	5	0	100%
System Testing	ST01 - ST05	5	5	0	100%
CUMULATIVE TOTAL	All 4 Levels	31	31	0	100%

Visual Application & STQA Test Verification Screenshots

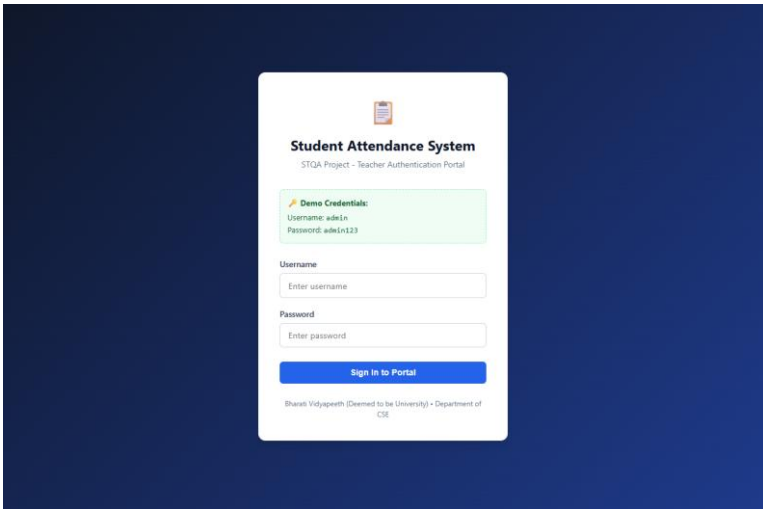


Figure 1 Teacher Authentication and Login Portal

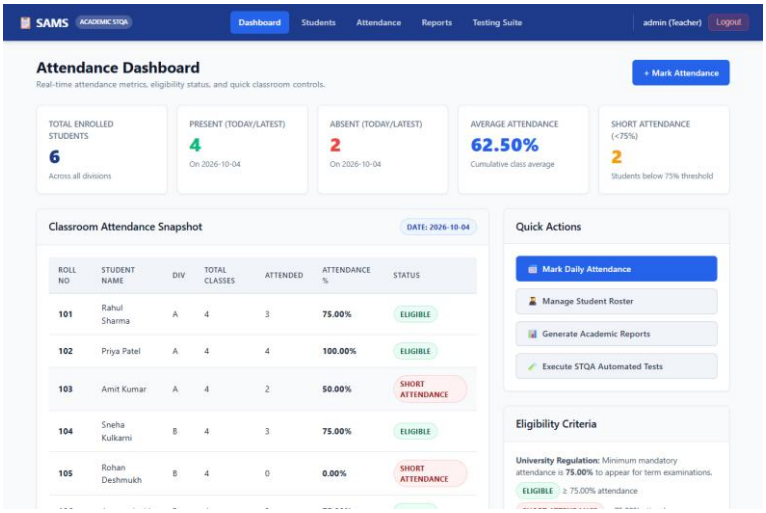


Figure 2 Attendance Dashboard and Classroom Metrics

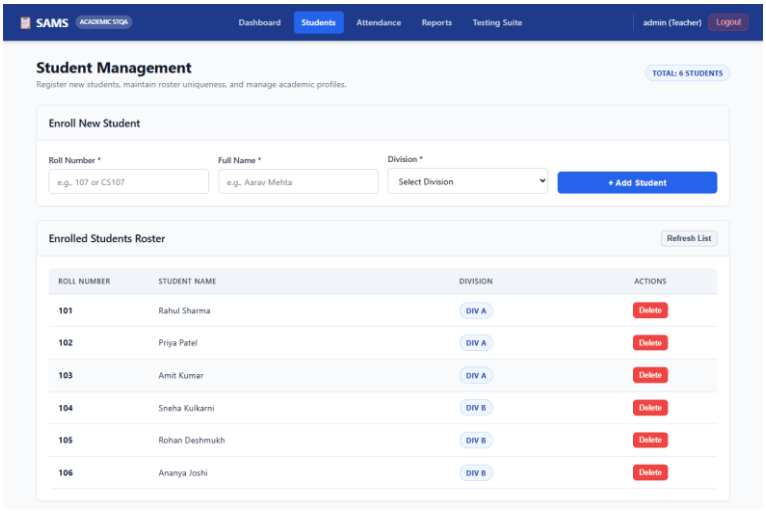


Figure 3 Student Management and Roster Registry

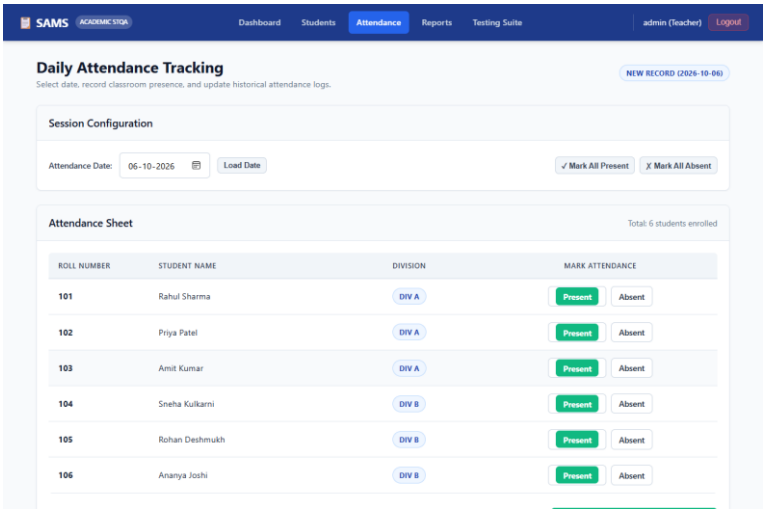


Figure 4 Daily Attendance Marking Interface

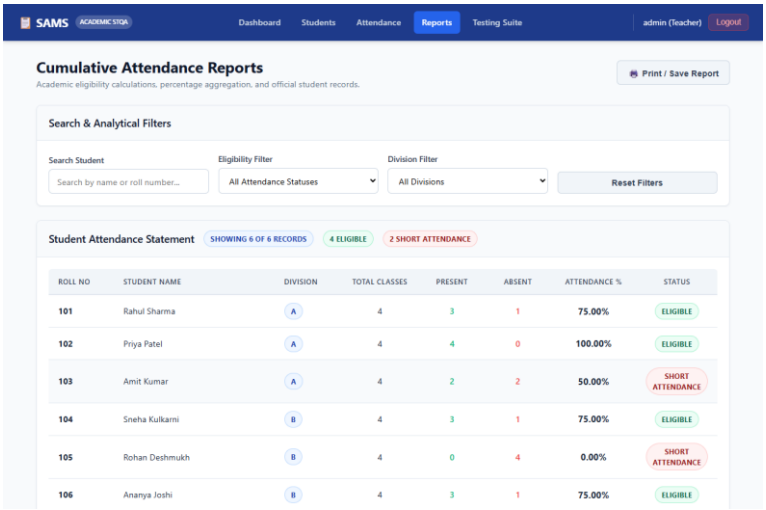


Figure 5 Cumulative Attendance Reports and Eligibility View

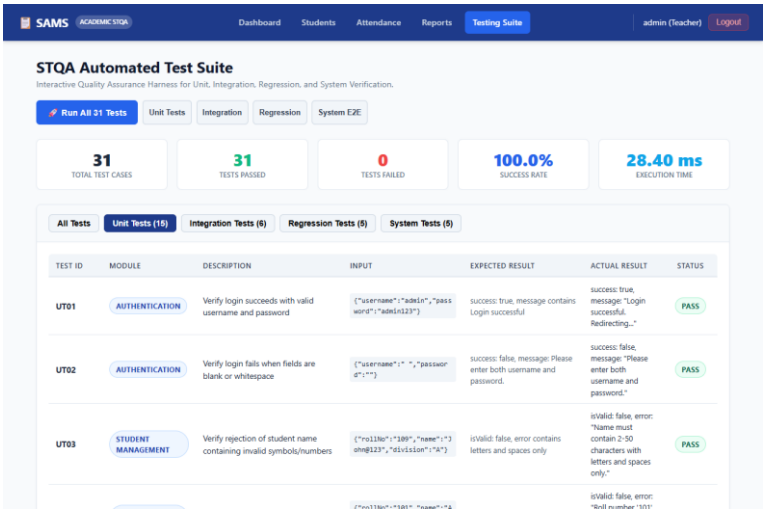


Figure 6 STQA Automated Unit Testing Suite Output

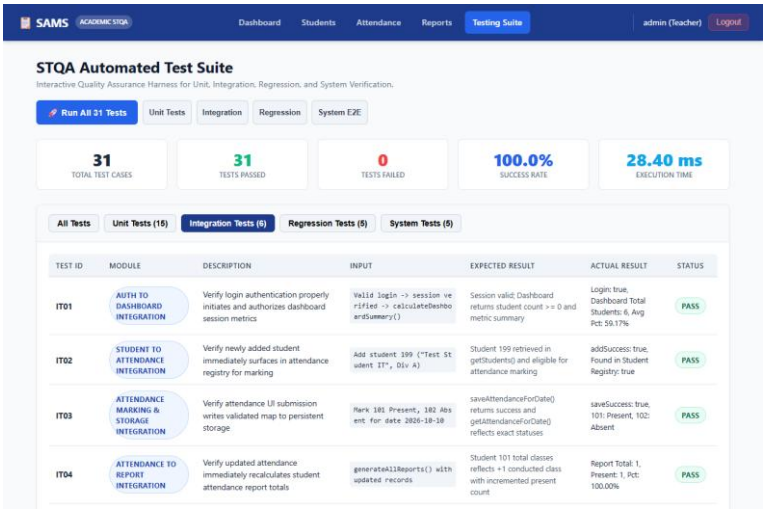


Figure 7 STQA Integration Testing Suite Output

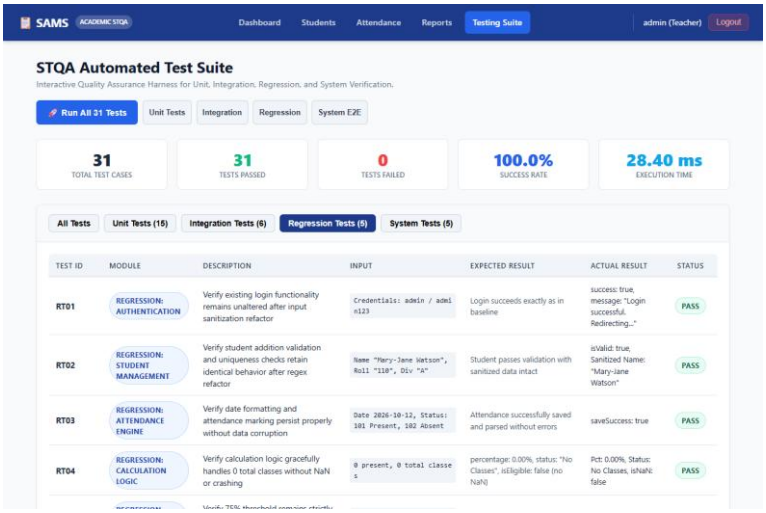


Figure 8 STQA Regression Testing Suite Output

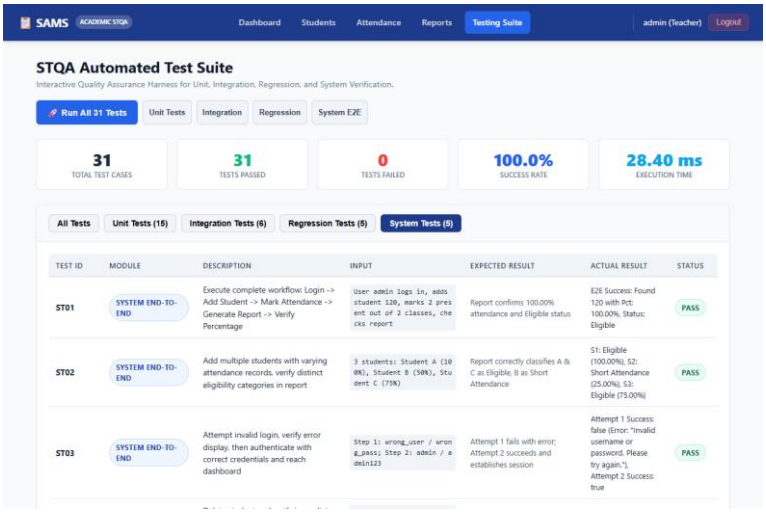


Figure 9 STQA System End-to-End Testing Output

Conclusion

Summary of Achievements

The Student Attendance Management System successfully fulfills all functional and STQA requirements. Built from scratch using vanilla web technologies, the application delivers a zero-configuration classroom utility. Automated execution of 31 test cases confirmed 100% pass rate with zero defect regressions. Boundary Value Analysis successfully demonstrated

that the 75% university eligibility rule behaves with complete mathematical fidelity (>= 75.00% Eligible, < 75.00% Short Attendance).

Future Scope

- **1. Role-Based Access Control:** Implement multi-role login for Teachers, HODs, and Students with granular view permissions.
- **2. Automated Hardware Integration:** Support contact-free attendance capture via RFID card readers or biometric scanners.
- **3. Automated Notifications:** Automate SMS/email alerts to parents when student attendance drops below 75%.

References & Bibliography

[1] G. J. Myers, C. Sandler, and T. Badgett, The Art of Software Testing, 3rd ed. Hoboken, NJ, USA: John Wiley & Sons, 2011.

[2] IEEE Standard for Developing a Software Project Life Cycle Process, IEEE Standard 1074-2006, 2006.

[3] IEEE Standard for Software and System Test Documentation, IEEE Standard 829-2008, 2008.

[4] P. C. Jorgensen, Software Testing: A Craftsman's Approach, 4th ed. Boca Raton, FL, USA: CRC Press, 2013.

[5] I. Sommerville, Software Engineering, 10th ed. Boston, MA, USA: Pearson, 2015.

[6] W3C, "HTML5: A Vocabulary and Associated APIs for HTML and XHTML," W3C Recommendation, 2014.